Data Preparation / Feature Engineering and Model Exploration

Project Title: NudgeWell: In-Session Intention–Behavior Gap Prediction for Digital Well-Being

Team Members (Group 6): Yusuf Ersel Kara • Bahadır Maral • Çağla Eryılmaz • Fatih Mehmet Menteşe • Damla Nur Öcal

Samsung Innovation Campus (SIC) AI, Cohort 18 · Digital Addiction Capstone, in partnership with Yeşilay
Instructor: Eda Nur Saruhan

Part I: Data Preparation & Feature Engineering

1. Overview

The data preparation and feature engineering phase is the analytical foundation of NudgeWell. Total screen time is a weak marker of harm: in a four-week logging study of 340 Android users, intense phone use alone did not predict negative well-being, and the two usage profiles linked with lower well-being were both marked by night-time sessions (Katevas et al., 2018). NudgeWell therefore looks at one specific moment inside a session, the in-session intention–behavior gap (IBG): the person reports that they should stop using the phone but keeps going (Zheng & Choe, 2026b).

This phase turns the AwareMind check-ins into a feature table that a phone could compute at the moment of a check-in. Three constraints shape it: (1) only information recorded before the prompt may enter a model; (2) missing values and heavy-tailed durations must be handled without discarding informative check-ins; and (3) people differ strongly: 39% of the variance of the continuous gap lies between participants (ICC = 0.39), so new users and known users have to be evaluated separately.

Phase objective: NudgeWell frames the task as in-session binary classification. A check-in is a *high-gap moment* when the stated intention is 5 or lower on the 1–10 scale (1 = stop, 10 = continue) and the person keeps using the phone for more than 5 minutes after answering. These are the thresholds of Zheng and Choe (2026b), who call them arbitrary and also analyse the gap as a continuous score. A model may use only what is known when the prompt appears.

2. Data Collection

NudgeWell uses the AwareMind dataset (Zheng & Choe, 2026a, 2026b). The authors' Android app logged phone use and, during sessions, asked “Reflecting on my phone usage, I should [ ] using my phone now”, answered on a 1–10 slider (1 = stop, 10 = continue), at most 16 times a day. The 37 participants were US adults (22 women, 15 men; aged 18–64, mean 42.1) who used an Android phone at least three hours a day and wanted to cut down. Of 48 people recruited, 6 dropped out for technical reasons and 5 were excluded for answering “continue” to more than 95% of the prompts.

Data were collected between 2 July and 6 October 2025; each participant took part for at least two weeks (14–32 days, median 19). The released file has 5,772 check-in rows and 92 columns from 5,549 sessions. After exact duplicates are removed (Section 3) it holds 5,627 check-ins, 72–294 per participant (median 148).

Data use. The OSF project carries no licence. We analyse the file for coursework only, do not redistribute it and keep it out of the public repository. Our concept note commits us to written reuse permission from the authors; until it is granted, the models in this report are exploratory and will not be published or deployed. The code also runs on a synthetic stand-in with the same 92 columns, built only from statistics printed in the paper (Part II, Section 4).

Scope and corpus justification. Two other public resources were considered:

- **Child Mind Institute (CMI) Problematic Internet Use** (Child Mind Institute, n.d.): questionnaire scores, physical-activity measures and wrist-worn accelerometer data from young people in the Healthy Brain Network study. Its label is a per-person severity score derived from the Parent-Child Internet Addiction Test. It has no session-level phone data and no in-the-moment intention, so it cannot train a model that acts during a session.
- **one sec** (Haliburton et al., 2024): usage logs from 1,039 users of a friction app over an average of 13.4 weeks, recording attempts to open target apps behind a short friction. It does not record whether the person wanted to stop.
- **AwareMind** (primary; Zheng & Choe, 2026a): the only public dataset that pairs a person's in-the-moment intention with what they did next, together with context recorded before the prompt (app category, time, session history, recent interactions and physical activity).

3. Data Cleaning

We audited all 92 columns of the released file. Four issues needed handling: duplicate rows, missing values, heavy-tailed durations and information recorded after the prompt.

Duplicate rows

145 rows are exact copies of another row: the same prompt identifier and identical values in all 92 columns. All of them belong to one participant, whose 290 rows are 145 check-ins recorded twice. We remove the copies, which leaves 5,627 unique check-ins. The paper's count of 5,772 includes them: with them the high-gap share is 14.9%, without them 12.8%. Every number below uses the de-duplicated data.

Handling missing values

Missingness was audited on the 5,627 unique check-ins. Rows are kept rather than deleted; deleting them would drop 15% of the data at most.

- **Foreground app** (`package_name`, `Category_L1`): missing in 703 check-ins (12.5%); the dataset does not record why. The app-category indicators stay 0 and a flag `app_missing` = 1 is added. The three app-duration features are missing in 700 rows; they are filled with the training-fold median inside the model pipeline, so no information from test participants is used.
- **Physical activity** (`activity_type`): missing in 127 check-ins (2.3%). The five activity indicators stay 0 and a flag `activity_missing` = 1 is added.
- **Interaction counts** (taps, scrolls, text edits): no missing values; 25.1% of check-ins have no recorded interaction. The paper analyses interactions “in the last minute” before the prompt. The data dictionary does not state the window, but the counts do not grow with session length (median taps: 2 in sessions shorter than 30 seconds, 0 in sessions longer than 10 minutes), so we treat them as recent-activity counts and do not divide them by session length.

Outliers and skew

The time a person keeps using the phone after answering is heavily right-skewed: median 5.0 minutes, maximum 646.6 minutes (about 11 hours). The binary label only compares it with 5 minutes, so it needs no capping. The dataset's continuous outcome caps it at 128 minutes (0.9% of check-ins) and log-transforms it (`remaining_log = log10(min(minutes, 128) + 1)`); we keep that definition for the continuous gap. Heavy-tailed predictors (durations, counts and scroll distances) are transformed with log(1 + x), which leaves tree models unchanged and stabilises the linear model.

Leakage control

Variables measured after the prompt would make the task trivial and cannot exist on a phone at prediction time. Table 1 lists everything that is removed. A guard in the pipeline stops the run if any outcome or identifier column re-enters the feature table.

Table 1: Variables removed before modelling.

Audit category	Removed variables	Reason
Outcomes measured after the prompt	`answer`, `remaining_time`, `remaining_minutes`, `remaining_capped`, `remaining_log`, `remaining_scaled`, `temp_scaled`, `intention_behavior_gap`, `lock_time`	Define or reveal the label; unknown when the prompt appears.
Exact copies	`session_length_till_esm` and `time_since_unlock` (both `session_minutes` in milliseconds); `screen_time_so_far` (copy of `screen_time_minutes`)	Redundant; copies split importance between them.
Identifiers and bookkeeping	`pid`, `PID`, `host`, `esm_id_full`, `session_id_combined`, `session_index`, `package_name`, `app_at_prompt`	Let a model memorise people or sessions; package names are replaced by app categories.
Raw timestamps	`date`, `date_4am`, `unlock_time`, `esm_time`, `esm_local`, `unlock_local`, `timezone`	Replaced by cyclic hour and weekday.
Static demographics (not leakage)	`Age`, `Gender_encoded`, `Education_encoded`, `Race_*` (5), `Employment_*` (7)	Constant within a person and unique for each of the 37 participants, so in a cross-user model they act as participant identifiers. Excluded from the main models and tested separately (Part II, Section 3).

We also checked whether any kept feature runs past the prompt. In 97 check-ins (three participants, almost all from one) the time spent in the current app during the session is longer than the time since unlock; the excess is unrelated to how long the person kept going, so we treat it as a device timing quirk, not leakage. The guard was also tested on synthetic data with nothing to find, where every model scored at chance (Part II, Section 4).

4. Exploratory Data Analysis (EDA)

Five findings shape the modelling choices (all on 5,627 unique check-ins; “pooled” means all check-ins counted together):

- **Prevalence.** 718 check-ins (12.8%) are high-gap moments. In 30.1% of check-ins people answered 10 (“continue”).
- **Night.** Between midnight and 06:00 the pooled share is 30.4% (n = 375) against 11.5% at other hours, peaking at 3 am (40%, n = 45; Figure 1). The effect holds within people: compared with their own daytime check-ins, participants' night-time share is 18.7 percentage points higher on

average (95% CI 8.9–30.1; higher at night for 16 of the 22 participants with at least five check-ins in both).

- **App category.** Pooled shares are highest for social media (19.9%), media and video (17.8%) and browsers (14.2%), and lowest for messaging (9.4%), work and study apps (8.5%) and system tools (3.7%) (Figure 2). Within people the differences shrink: a participant's social-media check-ins are 3.1 points above their other check-ins on average (95% CI −1.1 to 7.8; higher for only 11 of 27 participants). Part of the pooled difference therefore reflects *who* uses social media rather than what social media does.
- **Differences between people.** The share of high-gap moments ranges from 0% to 97% per participant (median 7.0%; Figure 3); three participants have none. The intraclass correlation is 0.39 for the continuous gap, 0.42 for the stated intention and 0.28 for the binary label. Women had a higher share than men (16.2% vs 8.1%).
- **Concentration.** One participant (97% high-gap) contributes 19.6% of all high-gap moments; the top three contribute 37.0% and the top ten 73.3%. Without that participant the pooled share is 10.5%. Pooled percentages therefore mix who is in the data with what happens within a person, so we report within-person comparisons where possible and evaluate models per participant.

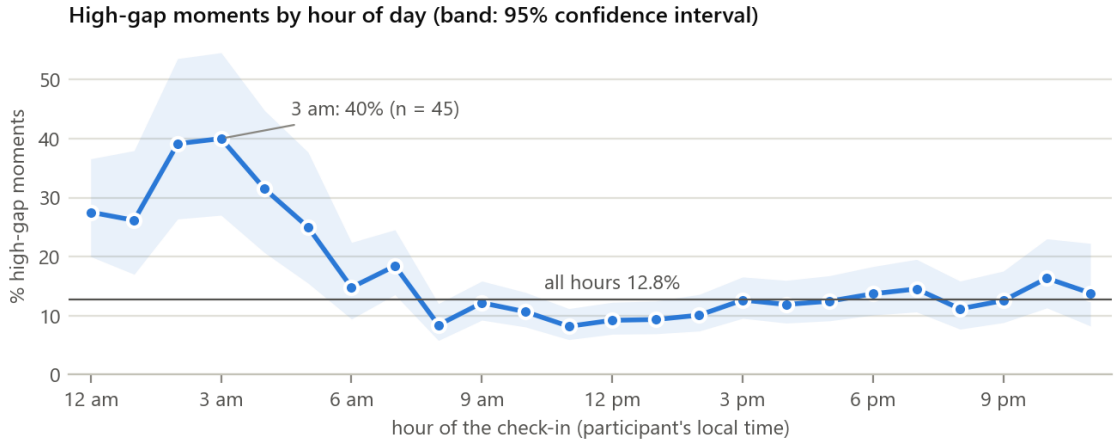


Figure 1: High-gap share by hour of the check-in (participant's local time), with 95% Wilson confidence band; 5,627 unique check-ins.

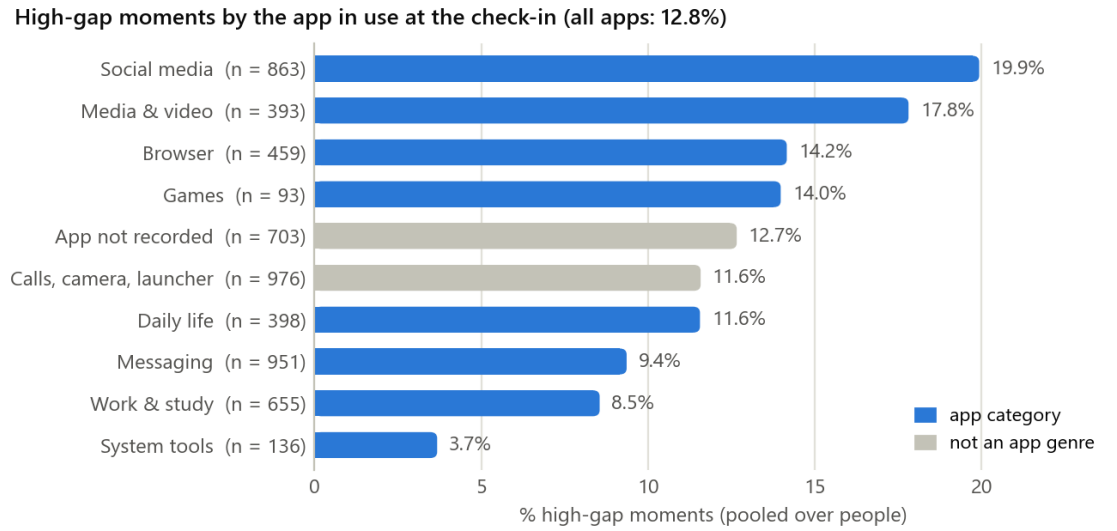


Figure 2: High-gap share by the app in use at the check-in, pooled over participants. "Calls, camera, launcher" is the dataset's "To exclude" group.

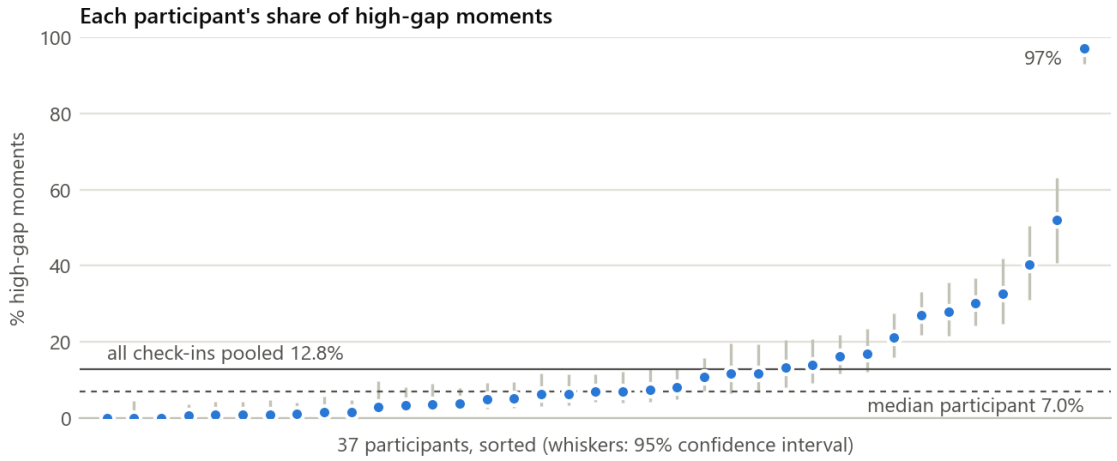


Figure 3: Each participant's share of high-gap moments, sorted, with 95% confidence intervals. The pooled share (12.8%) is well above the median participant (7.0%).

5. Feature Engineering

From the columns that pass the audit we build 49 features in five groups (Table 2). Every feature is computed from information available at the moment the prompt appears.

Table 2: Feature groups (counts in brackets).

Feature group	Engineered variables	Rationale
Time (10)	Sine and cosine of hour and weekday; weekend and workday flags; morning, afternoon, evening and night indicators	Cyclic encoding keeps 23:59 next to 00:01. Night is the strongest within-person signal.
Current session (5)	Session length so far; time in the current app (this visit and this session); distinct apps and app switches in the session	How long and how fragmented the session has been.
Earlier use today (8)	Screen time today; number and mean length of earlier sessions; length of the previous session and time since it ended; time in the current app today; distinct apps and app switches today	Heavy-use days and quick re-opening of the phone.
Recent interaction (10)	Taps, text edits, and scroll counts and distances in four directions	Passive scrolling versus active typing; the paper found opposite effects for scrolling and text editing.
App and activity (16)	Nine app-category indicators; five activity indicators (still, walking, running, cycling, in a vehicle); `app_missing` and `activity_missing`	What the person is doing and whether they are moving; missingness is made explicit.

Demographics are kept out of the main feature set because they identify participants (Table 1); Part II, Section 3 shows what happens when they are added.

6. Data Transformation

- **Log transform.** Durations, counts and scroll distances: $\log(1 + x)$, applied row by row.
- **Imputation.** The three app-duration features: median of the training fold, learned inside the model pipeline.
- **Scaling.** Standardisation (z-scores) only for logistic regression, fitted inside each training fold; the tree models need none.

- **Categorical variables.** App category, physical activity and time period are one-hot indicators, as released. No ordinal codes are used: the coding key of `Education_encoded` is not documented, and age is a continuous variable (the data have no age bands).

The feature-building step of our pipeline (`code/hw4_pipeline.py`):

```
def build_features(df, demographics=False):
    """One row per check-in, using only what is known when the prompt appears."""
    X = pd.DataFrame(index=df.index)
    angle_h = 2 * np.pi * df["esm_hour"] / 24  # cyclic: 23:59 sits next to 00:01
    angle_d = 2 * np.pi * df["esm_weekday"] / 7
    X["hour_sin"], X["hour_cos"] = np.sin(angle_h), np.cos(angle_h)
    X["weekday_sin"], X["weekday_cos"] = np.sin(angle_d), np.cos(angle_d)
    for c in TIME[4:] + SESSION + TODAY + INTERACTION + GENRES + ACTIVITY:
        X[c] = df[c].astype(float)
    X["app_missing"] = df["Category_L1"].isna().astype(float)  # not recorded
    X["activity_missing"] = df["activity_type"].isna().astype(float)  # not recorded
    X[LOG1P] = np.log1p(X[LOG1P].clip(lower=0))
    if demographics:
        X[DEMOGRAPHICS] = df[DEMOGRAPHICS].astype(float)
    # Leakage guard: fails if an outcome or identifier column ever enters the table.
    assert not set(X.columns) & set(OUTCOME + IDENTIFIERS), "leakage guard"
    return X[FEATURES + (DEMOGRAPHICS if demographics else [])]
```

Part II: Model Exploration

1. Model Selection

This milestone compares tabular baselines. The project's main model, a compact temporal convolutional network (TCN; Bai et al., 2018) that reads each person's earlier check-ins in time order, is developed in the next milestone, as set out in our concept note; the baselines below define the bar it must clear. Candidates were judged on four criteria: (1) capacity for non-linear interactions; (2) behaviour under class imbalance (12.8% positives); (3) size and speed on a phone, because keeping behavioural data on the device supports the data-minimisation principle of Law No. 6698 on the Protection of Personal Data (Law No. 6698, 2016); and (4) whether the model can be adapted to one person without sending data off the phone.

Table 3: Candidate models.

Model	Strengths	Weaknesses	On-device use
Logistic regression (L2)	Interpretable coefficients; tiny; stable probabilities	Only additive effects unless interactions are added	A few hundred bytes of weights
Random forest (Breiman, 2001)	Captures interactions; insensitive to scaling; out-of-bag error estimate	Cannot be updated incrementally; larger than a linear model	100 trees of depth ≤ 6: about 9,000 nodes, under 1 MB; inference only, so personal updates need retraining
XGBoost (Chen & Guestrin, 2016)	Regularised gradient boosting; strong on tabular data; handles missing values natively	Overfits small samples unless depth and learning rate are limited	Similar to the random forest; inference only
LightGBM (Ke et al., 2017)	Fast histogram-based boosting with leaf-wise trees; handles missing values natively	Leaf-wise growth overfits small data without a depth limit	Similar to the random forest; inference only
Temporal convolutional network (next milestone)	Uses the order of earlier check-ins	More tuning; must beat the baselines on the same splits	LiteRT or ONNX Runtime Mobile; personal re-calibration on the device

Size and speed on a phone are targets (under 5 MB, under 10 ms per prediction) to be measured on a device in the next milestone; they are not measured here.

2. Model Training

Two evaluation regimes answer two different questions:

- **A. New users (global).** Leave-one-participant-out (LOPO) cross-validation, the protocol of Zheng and Choe (2026b): each participant is scored by a model trained on the other 36. We report pooled metrics over all held-out check-ins with 95% confidence intervals from a bootstrap over participants, and the within-person ROC-AUC (how well the model ranks one person's own check-ins), averaged over the 34 participants who have both classes.
- **B. Known users (personal).** Each participant's check-ins are put in time order; the first 70% are used for training and the last 30% for testing, for all 37 participants. Every approach is scored on the same 1,705 test check-ins: (i) the person's own base rate, their training-slice share shrunk toward the population rate; (ii) the global model with no personal data; (iii) the global model plus a personal intercept, one number per person fitted on the training slice; (iv) a personal random forest and (v) a personal XGBoost model, each trained only on the person's own data; and (vi) a combined random forest trained on everyone else plus the person's training slice, weighted five times.

No participant is excluded on the basis of the outcome. Within-person metrics can only be computed for participants whose test slice contains both classes; they are reported for that group with its own chance level.

Hyperparameters and class imbalance

- **Logistic regression:** L2 penalty, C = 1.0, class_weight = 'balanced', max_iter = 2000.
- **Random forest:** n_estimators = 100, max_depth = 6, min_samples_split = 10, class_weight = 'balanced'.
- **XGBoost:** n_estimators = 100, max_depth = 4, learning_rate = 0.05, scale_pos_weight = negatives / positives of the training fold (about 6.8).
- **LightGBM:** n_estimators = 100, max_depth = 5, num_leaves = 31, learning_rate = 0.05, class_weight = 'balanced'.

All settings were fixed before testing; nothing was tuned on test participants. Class weighting counters the 12.8% prevalence but inflates the predicted probabilities, so the Brier scores below are reported for transparency; calibration (Platt scaling or isotonic regression; Niculescu-Mizil & Caruana, 2005) is planned for the next milestone.

3. Model Evaluation

ROC-AUC measures ranking, with 0.5 as chance. PR-AUC focuses on the high-gap class; its chance level equals the prevalence of the rows being evaluated, not a fixed number (Saito & Rehmsmeier, 2015). The Brier score measures probability error; predicting the prevalence for everyone gives 0.111. We also compare the models with the fixed rule from our concept note, which alerts when it is night, a social-media app is open or the session has lasted more than 10 minutes, at the same alert rate (24.4% of check-ins).

Table 4: New users (leave-one-participant-out, 37 folds, 5,627 check-ins).

Model	Pooled ROC-AUC (95% CI)	PR-AUC (chance 0.128)	Brier (constant 0.111)	Within-person ROC-AUC (95% CI)	Precision / recall at 24.4% alerts
Fixed rule (no training)	0.60 (0.55–0.65)	0.17	—	0.57 (0.53–0.61)	21.8% / 41.8%
Logistic regression	0.62 (0.55–0.67)	0.18	0.223	0.62 (0.55–0.68)	18.0% / 34.4%
Random forest	0.60 (0.55–0.65)	0.16	0.193	0.63 (0.58–0.68)	16.4% / 31.3%
XGBoost	0.57 (0.52–0.63)	0.15	0.197	0.61 (0.56–0.66)	16.7% / 31.9%
LightGBM	0.56 (0.50–0.62)	0.15	0.188	0.60 (0.55–0.65)	17.0% / 32.6%
Random forest + demographics	0.56 (0.49–0.64)	0.15	0.176	0.63 (0.57–0.69)	16.7% / 32.0%
Logistic regression + demographics	0.52 (0.43–0.61)	0.14	0.223	0.63 (0.57–0.69)	14.3% / 27.4%

- **Modest signal for new users.** The best model, logistic regression, reaches a pooled ROC-AUC of 0.62 (95% CI 0.55–0.67); the random forest 0.60, XGBoost 0.57, LightGBM 0.56. Within a person, all four rank high-gap moments above chance (0.60–0.63; above 0.5 for 79%–85% of participants).
- **A simple rule does as well.** The three-signal rule has about the same pooled ROC-AUC as the random forest (0.60) and, at its own alert rate, higher precision and recall (21.8% and 41.8%) than the best model (18.0% and 34.4%; Figure 6). For new users these baselines do not yet justify a learned model; the temporal model has to beat this rule (concept note, objective O4).
- **Demographics act as identifiers.** Adding the 15 demographic columns lowers the pooled ROC-AUC (random forest 0.60 → 0.56; logistic regression 0.62 → 0.52) while the within-person ROC-AUC stays the same. When included, they take 47% of the forest's Gini importance, led by age, education and retirement. Under a 5-fold participant split, the same forest scores 0.62 ± 0.03 without them and 0.58 ± 0.10 with them.
- **Calibration.** Every model has a Brier score above the constant prediction (0.111) because of class weighting; probabilities must be calibrated before they drive a decision.

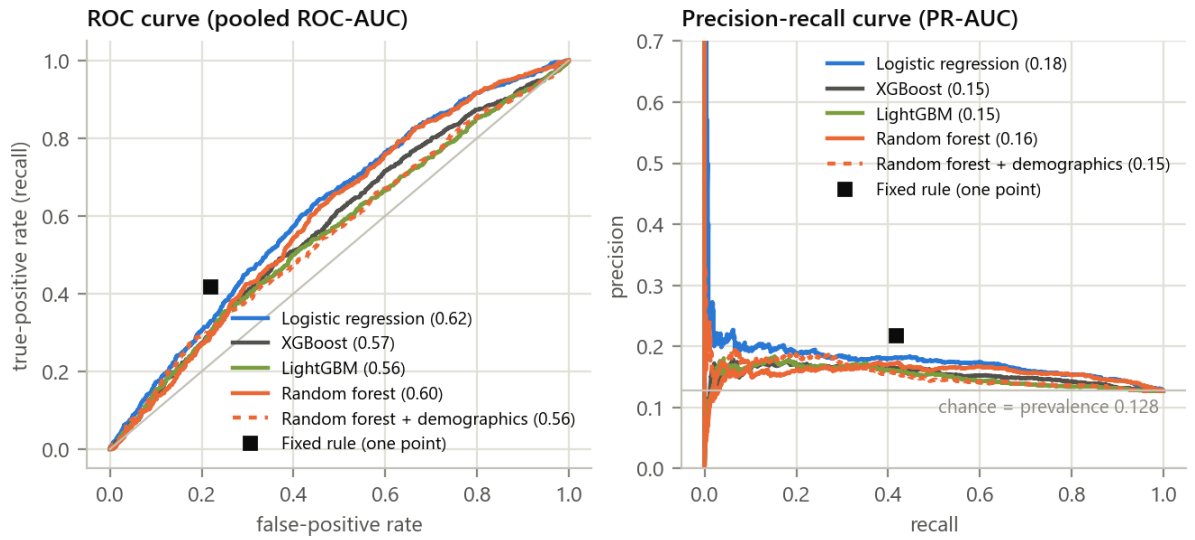


Figure 4: New users (leave-one-participant-out): ROC and precision-recall curves of the pooled held-out predictions; the legend gives the pooled AUCs. The square marks the fixed rule. Chance is the diagonal (ROC) and the prevalence line (PR).

Table 5: Known users (first 70% of each participant's check-ins for training, last 30% for testing; 1,705 test check-ins). Within-person metrics cover the 25 participants whose test slice has both classes.

Approach	Pooled ROC-AUC (95% CI)	PR-AUC (chance 0.128)	Brier	Within-person ROC-AUC (95% CI)	Within-person PR-AUC (chance 0.26)
Own base rate (one number, no model)	0.83 (0.74–0.90)	0.51	0.084	0.50 (0.50–0.50)	0.20
Global random forest (no personal data)	0.60 (0.53–0.66)	0.15	0.193	0.60 (0.53–0.67)	0.31
Global + personal intercept	0.84 (0.75–0.90)	0.54	0.082	0.60 (0.53–0.67)	0.31
Personal random forest (own data only)	0.84 (0.75–0.91)	0.56	0.086	0.59 (0.49–0.68)	0.36
Personal XGBoost (own data only)	0.81 (0.74–0.87)	0.37	0.109	0.61 (0.53–0.69)	0.37
Combined random forest (others + own ×5)	0.74 (0.63–0.83)	0.41	0.170	0.62 (0.52–0.70)	0.37

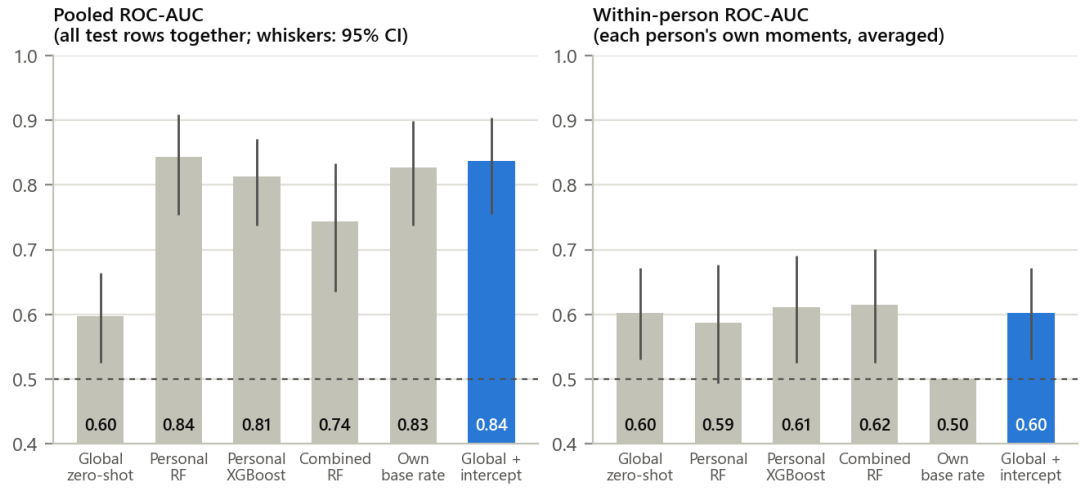


Figure 5: Known users: every approach scored on the same last 30% of each participant's check-ins. Left: pooled ROC-AUC (finding the right people); right: within-person ROC-AUC (finding the right moments). Whiskers: 95% bootstrap intervals over participants.

- **Knowing the person is the big gain.** One number per person, their own share of high-gap moments in the training slice, lifts the pooled ROC-AUC from 0.60 (global model) to 0.83. People's rates are stable over time: the correlation between a person's share in the first 70% and the last 30% of their check-ins is 0.87.
- **Global model + personal intercept ≈ personal model.** Adding that one number to the global model gives a pooled ROC-AUC of 0.84 and a PR-AUC of 0.54, close to a forest trained on the person's own data (0.84 and 0.56) and above a personal XGBoost model (0.81 and 0.37), and it needs no training on the phone.
- **Ranking a person's own moments is equally hard for every approach.** The within-person ROC-AUC is 0.60 for the global model, 0.59 for the personal forest and 0.61 for the personal XGBoost model (personal forest minus global: −0.01, 95% CI −0.10 to +0.07; the personal forest is better for 10 of 25 participants). With two to four weeks of data, personal data tells us *who* is at risk, not yet *when*.

- **Chance levels differ by regime.** The participants with both classes in their test slice have a mean prevalence of 0.199, and random scores on exactly their rows give a within-person PR-AUC of 0.26. PR-AUC values from different regimes can therefore only be compared against their own chance level.

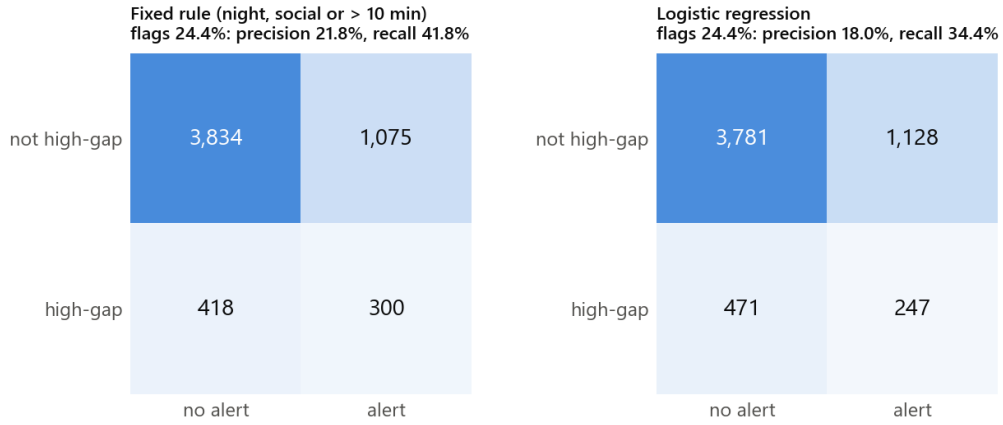


Figure 6: Confusion matrices when the fixed rule and the best model each flag 24.4% of check-ins (new users, leave-one-participant-out).

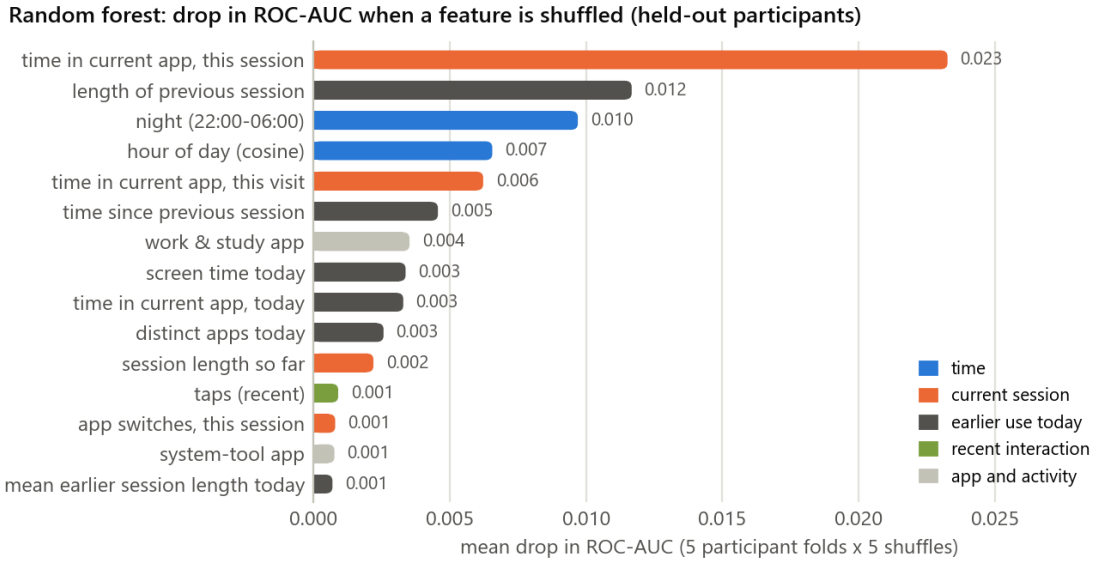


Figure 7: Random forest without demographics: drop in ROC-AUC when one feature is shuffled, measured on held-out participants (5-fold participant split, 5 shuffles). Session durations, the previous session and night carry most of the signal; recent-interaction features add none for unseen people.

What this means for the next milestone

- The temporal model must beat two baselines on the same splits: the fixed rule at its alert rate (precision 21.8%, recall 41.8%) for new users, and the global model with a personal intercept for known users (pooled ROC-AUC 0.84).
- Personalisation should start as a per-person intercept on top of a global model: one number, updated on the phone as check-ins accumulate. A separately trained personal model adds nothing measurable with this amount of data.
- Probabilities must be calibrated before a decision threshold and a daily prompt budget are fixed.

- **Limitations.** 37 US adults followed for 14–32 days; one participant contributes 20% of all high-gap moments; the binary label uses thresholds the dataset authors call arbitrary; and the results are exploratory until reuse permission is granted.

4. Code Implementation

The full pipeline is `code/hw4_pipeline.py` in our repository. It reads the AwareMind file or the synthetic stand-in, removes duplicates, builds the 49 features, runs both evaluation regimes and writes every number in this report to `results.json`; the figures and this document are generated from that file. The key parts follow (the feature step is in Part I, Section 6).

```
# Measured after the prompt: they define or reveal the label, never model inputs.
OUTCOME = ["answer", "remaining_time", "remaining_minutes", "remaining_capped",
           "remaining_log", "remaining_scaled", "temp_scaled",
           "intention_behavior_gap", "lock_time"]

# Identifiers, bookkeeping and raw timestamps (memorise people or sessions).
IDENTIFIERS = ["pid", "PID", "host", "esm_id_full", "session_id_combined",
               "session_index", "package_name", "app_at_prompt", "date", "date_4am",
               "unlock_time", "esm_time", "esm_local", "unlock_local", "timezone",
               "is_synthetic"]

def load(path):
    """Read the file, drop exact duplicate rows and add the label."""
    raw = pd.read_csv(path, low_memory=False)
    df = raw.drop_duplicates().reset_index(drop=True)
    # High-gap: wanted to stop (answer 1-5 of 10) but used the phone > 5 more minutes.
    df["high_gap"] = ((df["answer"] <= 5) & (df["remaining_minutes"] > 5)).astype(int)
    return raw, df

def lopo(model, X, y, groups):
    """New users: each participant is scored by a model trained on everyone else."""
    oof = np.zeros(len(y))
    for g in np.unique(groups):
        te = groups == g
        oof[te] = clone(model).fit(X[~te], y[~te]).predict_proba(X[te])[:, 1]
    return oof

def chrono_split(df, frac=0.7):
    """Known users: in time order, each person's first 70% of check-ins train,
    the last 30% test."""
    train = np.zeros(len(df), bool)
    for _, idx in df.groupby("pid").indices.items():
        order = idx[np.argsort(df["esm_time"].values[idx], kind="stable")]
        train[order[: int(frac * len(order))]] = True
    return train

def fit_offset(s, y, a0=0.0, tau=None, iters=50):
    """Intercept a in p = expit(a + logit(s)) by Newton steps; an optional
    normal prior N(a0, tau^2) shrinks it toward a0 when a person has few check-ins."""
    z = logit(np.clip(s, 1e-4, 1 - 1e-4))
    a = a0
    for _ in range(iters):
        p = expit(a + z)
        g = np.sum(y - p) - (0 if tau is None else (a - a0) / tau**2)
        h = -np.sum(p * (1 - p)) - (0 if tau is None else 1 / tau**2)
        a -= g / h
    return a
```

In the personal regime, each participant's intercept is fitted on their own training slice with `fit_offset(global_scores, labels, a0=population_intercept, tau=1.0)` and applied to the global model's scores on their test slice; the prior keeps the intercept close to the population value when a person has few check-ins.

Pipeline checks on synthetic data

Because the real file cannot be shared, the code was also run on the project's synthetic stand-in (`simulator/`). It has the same 92 columns but is generated only from statistics printed in the paper, with effects that can be switched off. Its numbers test the code; they are not findings about people.

Table 6: Pipeline checks on synthetic data (5-fold participant split).

Check	What it tests	Result (ROC-AUC, mean ± SD over 6 datasets)
NULL mode (no planted effects)	Nothing to find: a pipeline that scores above chance has a bug or a leak	Logistic regression 0.50 ± 0.01; random forest 0.49 ± 0.01; XGBoost 0.50 ± 0.01; LightGBM 0.50 ± 0.01
Default mode (effects from the paper)	Planted effects are recovered	Logistic regression 0.61 ± 0.03; random forest 0.61 ± 0.03; XGBoost 0.60 ± 0.03; LightGBM 0.59 ± 0.03
Full run (seed 2410)	Both regimes run end to end without the real file	Completed; all results written to results.json

The stand-in has less person-level variation than the real data (intraclass correlation of the binary label 0.10 against 0.28) and no extreme participant, so it is suitable for testing the code but not for studying personalisation.

References

Bai, S., Kolter, J. Z., & Koltun, V. (2018). *An empirical evaluation of generic convolutional and recurrent networks for sequence modeling* (arXiv:1803.01271). arXiv. https://doi.org/10.48550/arXiv.1803.01271

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). ACM. https://doi.org/10.1145/2939672.2939785

Child Mind Institute. (n.d.). *Child Mind Institute — Problematic internet use* [Competition data set]. Kaggle. Retrieved September 19, 2026, from https://www.kaggle.com/competitions/child-mind-institute-problematic-internet-use

Haliburton, L., Grüning, D. J., Riedel, F., Schmidt, A., & Terzimehić, N. (2024). A longitudinal in-the-wild investigation of design frictions to prevent smartphone overuse. In *Proceedings of the CHI Conference on Human Factors in Computing Systems* (pp. 1–16). ACM. https://doi.org/10.1145/3613904.3642370

Katevas, K., Arapakis, I., & Pielot, M. (2018). Typical phone use habits: Intense use does not predict negative well-being. In *Proceedings of the 20th International Conference on Human-Computer Interaction with Mobile Devices and Services* (pp. 1–13). ACM. https://doi.org/10.1145/3229434.3229441

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30* (pp. 3146–3154). Curran Associates. https://proceedings.neurips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html

Law No. 6698 on the Protection of Personal Data. (2016). *Official Gazette of the Republic of Türkiye*, No. 29677 (7 April 2016). English translation retrieved October 11, 2026, from https://www.kvkk.gov.tr/Icerik/6649/Personal-Data-Protection-Law

Niculescu-Mizil, A., & Caruana, R. (2005). Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning* (pp. 625–632). ACM. https://doi.org/10.1145/1102351.1102430

Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE, 10*(3), e0118432. https://doi.org/10.1371/journal.pone.0118432

Zheng, J., & Choe, E. K. (2026a). *Intention-behavior gap in smartphone usage* [Data set]. OSF. https://osf.io/x6yzs/

Zheng, J., & Choe, E. K. (2026b). *To stop or not to stop: Exploring the intention-behavior gaps in smartphone usage* (arXiv:2609.08909) [Preprint, MobileHCI 2026]. arXiv. https://doi.org/10.48550/arXiv.2609.08909